



Loan Approval Prediction

This kernel I want try to make a loan approval prediction based on [Loan Prediction](#) dataset. Previously, I was made the same work for my university couse task but with different dataset. You can see my previous work on [this link](#) (in Bahasa). This work will compare the result of loan approval prediction classification algorithm with different dataset from my previous work. In previous work I get the Gradient Boosting Classifier as the best classifier for loan aproval prediction. Beside that, I want to make some improvement in this kernel inspired by the work of [Baligh's](#).

1. Import Packages & Data

```
In [1]: #Import packages
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.ensemble import GradientBoostingClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import cross_val_score
from sklearn.tree import DecisionTreeClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn import svm

#Read CSV data
data = pd.read_csv("../input/train_u6lujuX_CVtuZ9i (1).csv")
```

```
In [2]: #preview data
data.head()
```

```
Out[2]:
```

	Loan_ID	Gender	Married	Dependents	Education	Self_Employed	Applicant
0	LP001002	Male	No	0	Graduate	No	
1	LP001003	Male	Yes	1	Graduate	No	
2	LP001005	Male	Yes	0	Graduate	Yes	
3	LP001006	Male	Yes	0	Not Graduate	No	
4	LP001008	Male	No	0	Graduate	No	

2. Data Quality & Missing Value Assesment

```
In [3]: #Preview data information
data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 614 entries, 0 to 613
Data columns (total 13 columns):
Loan_ID          614 non-null object
Gender           601 non-null object
Married          611 non-null object
Dependents       599 non-null object
Education        614 non-null object
Self_Employed    582 non-null object
ApplicantIncome  614 non-null int64
CoapplicantIncome 614 non-null float64
LoanAmount       592 non-null float64
Loan_Amount_Term 600 non-null float64
Credit_History   564 non-null float64
Property_Area     614 non-null object
Loan_Status       614 non-null object
dtypes: float64(4), int64(1), object(8)
memory usage: 62.4+ KB
```

```
In [4]: #Check missing values
data.isnull().sum()
```

```
Out[4]: Loan_ID          0
Gender           13
Married          3
Dependents       15
Education        0
Self_Employed    32
ApplicantIncome   0
CoapplicantIncome 0
LoanAmount       22
Loan_Amount_Term  14
Credit_History   50
Property_Area     0
Loan_Status       0
dtype: int64
```

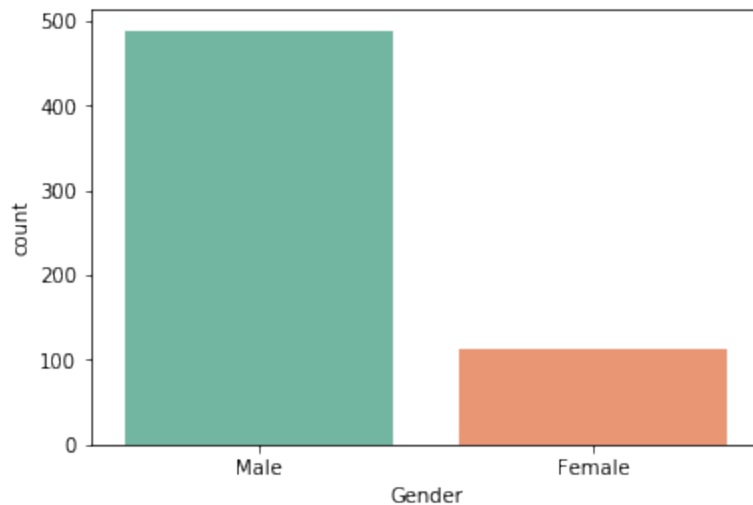
Gender - Missing Values

```
In [5]: # percent of missing "Gender"
print('Percent of missing "Gender" records is %.2f%%' %((data['Gender'].isnull()
Percent of missing "Gender" records is 2.12%
```

```
In [6]: print("Number of people who take a loan group by gender :")
print(data['Gender'].value_counts())
sns.countplot(x='Gender', data=data, palette = 'Set2')
```

```
Number of people who take a loan group by gender :
Male      489
Female    112
Name: Gender, dtype: int64
```

```
Out[6]: <matplotlib.axes._subplots.AxesSubplot at 0x7f4f45cd49b0>
```



Married - Missing Values

```
In [7]: # percent of missing "Married"  
print('Percent of missing "Married" records is %.2f%%' % ((data['Married'].isnu
```

Percent of missing "Married" records is 0.49%

```
In [8]: print("Number of people who take a loan group by marital status :")  
print(data['Married'].value_counts())  
sns.countplot(x='Married', data=data, palette = 'Set2')
```

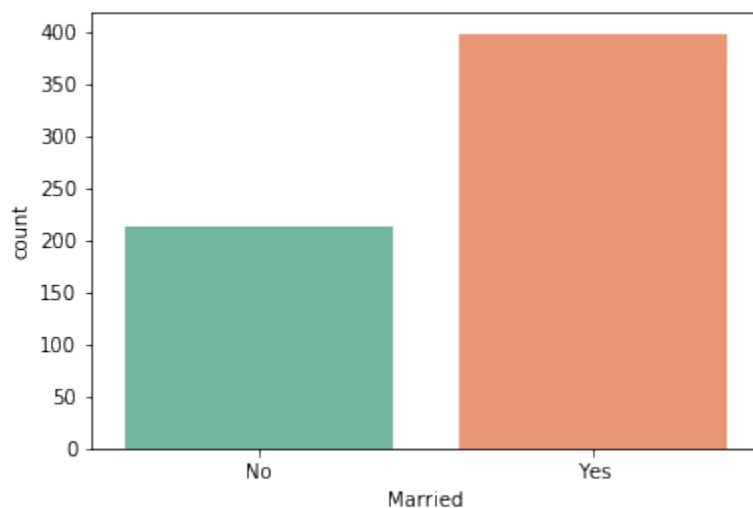
Number of people who take a loan group by marital status :

Yes 398

No 213

Name: Married, dtype: int64

Out[8]: <matplotlib.axes._subplots.AxesSubplot at 0x7f4f423df630>



Dependents- Missing Values

```
In [9]: # percent of missing "Dependents"
print('Percent of missing "Dependents" records is %.2f%%' %((data['Dependents'
```

Percent of missing "Dependents" records is 2.44%

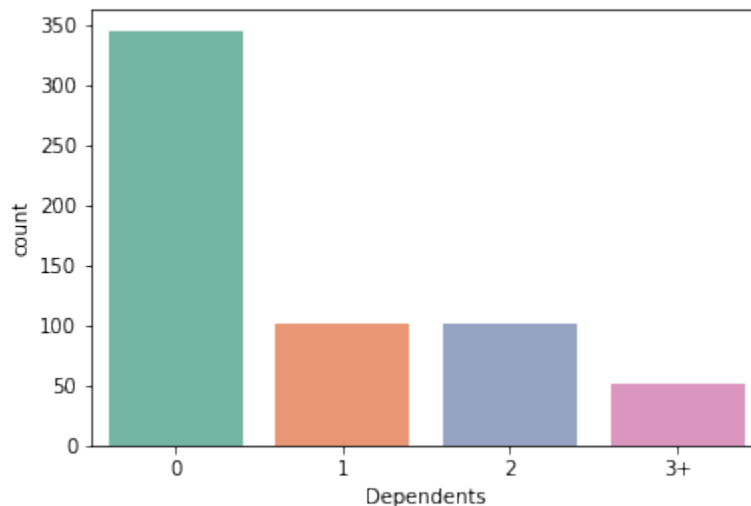
```
In [10]: print("Number of people who take a loan group by dependents :")
print(data['Dependents'].value_counts())
sns.countplot(x='Dependents', data=data, palette = 'Set2')
```

Number of people who take a loan group by dependents :

0	345
1	102
2	101
3+	51

Name: Dependents, dtype: int64

Out[10]: <matplotlib.axes._subplots.AxesSubplot at 0x7f4f423d1c50>



Self Employed - Missing Values

```
In [11]: # percent of missing "Self_Employed"
print('Percent of missing "Self_Employed" records is %.2f%%' %((data['Self_Emp
```

Percent of missing "Self_Employed" records is 5.21%

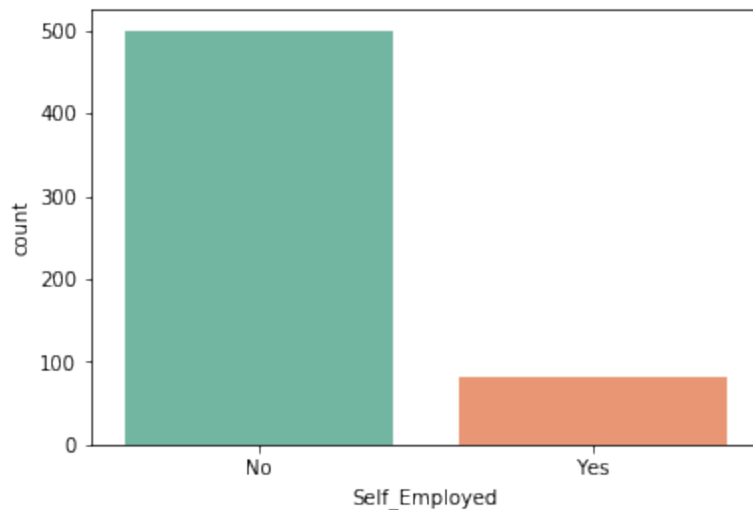
```
In [12]: print("Number of people who take a loan group by self employed :")
print(data['Self_Employed'].value_counts())
sns.countplot(x='Self_Employed', data=data, palette = 'Set2')
```

Number of people who take a loan group by self employed :

No	500
Yes	82

Name: Self_Employed, dtype: int64

Out[12]: <matplotlib.axes._subplots.AxesSubplot at 0x7f4f42332be0>

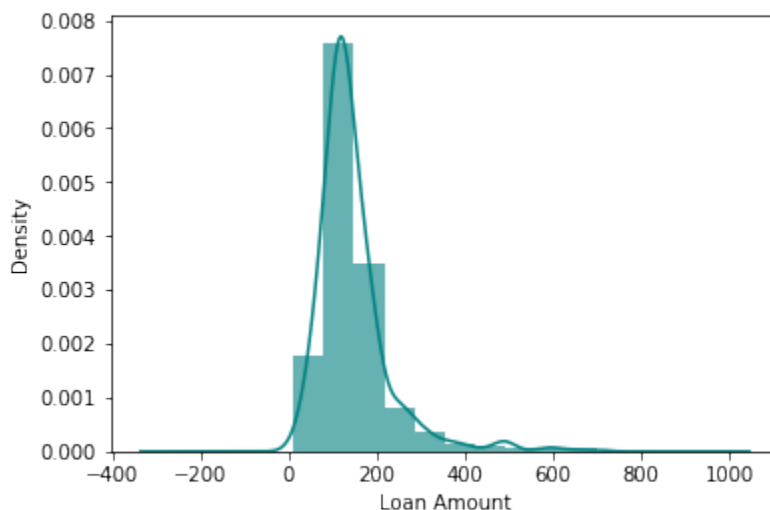


Loan Amount - Missing Values

```
In [13]: # percent of missing "LoanAmount"
print('Percent of missing "LoanAmount" records is %.2f%%' % ((data['LoanAmount']
```

Percent of missing "LoanAmount" records is 3.58%

```
In [14]: ax = data["LoanAmount"].hist(density=True, stacked=True, color='teal', alpha=0.5)
data["LoanAmount"].plot(kind='density', color='teal')
ax.set(xlabel='Loan Amount')
plt.show()
```



Loan Amount Term - Missing Values

```
In [15]: # percent of missing "Loan_Amount_Term"
print('Percent of missing "Loan_Amount_Term" records is %.2f%%' % ((data['Loan_
```

Percent of missing "Loan_Amount_Term" records is 2.28%

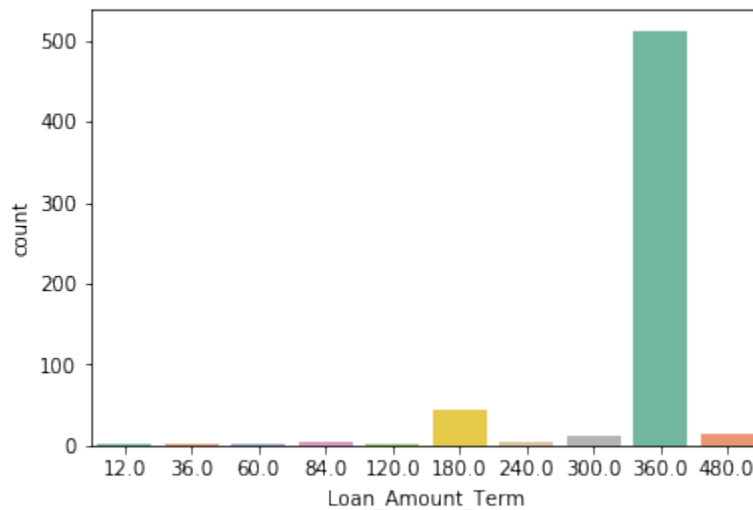
```
In [16]: print("Number of people who take a loan group by loan amount term :")
print(data['Loan_Amount_Term'].value_counts())
sns.countplot(x='Loan_Amount_Term', data=data, palette = 'Set2')
```

Number of people who take a loan group by loan amount term :

360.0	512
180.0	44
480.0	15
300.0	13
84.0	4
240.0	4
120.0	3
36.0	2
60.0	2
12.0	1

Name: Loan_Amount_Term, dtype: int64

Out[16]: <matplotlib.axes._subplots.AxesSubplot at 0x7f4f4226bc18>



Credit History - Missing Values

```
In [17]: # percent of missing "Credit_History"
print('Percent of missing "Credit_History" records is %.2f%%' % ((data['Credit_
```

Percent of missing "Credit_History" records is 8.14%

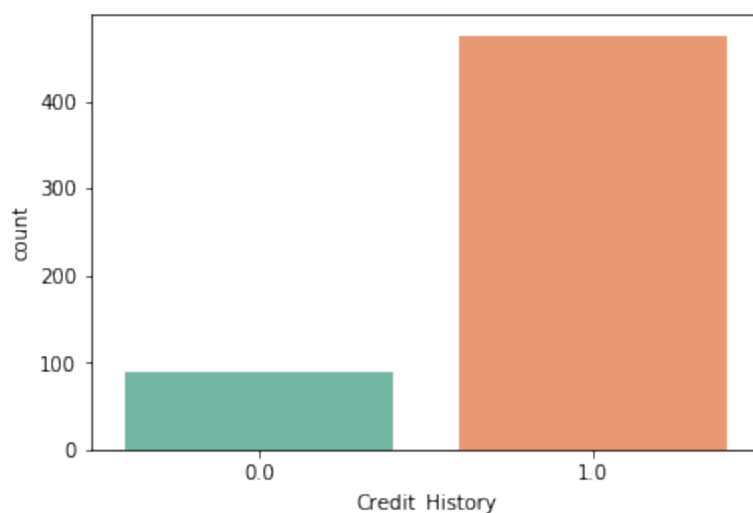
```
In [18]: print("Number of people who take a loan group by credit history :")
print(data['Credit_History'].value_counts())
sns.countplot(x='Credit_History', data=data, palette = 'Set2')
```

Number of people who take a loan group by credit history :

1.0	475
0.0	89

Name: Credit_History, dtype: int64

Out[18]: <matplotlib.axes._subplots.AxesSubplot at 0x7f4f422ec668>



3. Final Adjustments to Data

Based on my assessment of the missing values in the dataset, I'll make the following changes to the data:

- If "Gender" is missing for a given row, I'll impute with Male (most common answer).
- If "Married" is missing for a given row, I'll impute with yes (most common answer).
- If "Dependents" is missing for a given row, I'll impute with 0 (most common answer).
- If "Self_Employed" is missing for a given row, I'll impute with no (most common answer).
- If "LoanAmount" is missing for a given row, I'll impute with mean of data.
- If "Loan_Amount_Term" is missing for a given row, I'll impute with 360 (most common answer).
- If "Credit_History" is missing for a given row, I'll impute with 1.0 (most common answer).

```
In [19]: train_data = data.copy()
train_data['Gender'].fillna(train_data['Gender'].value_counts().idxmax(), inplace=True)
train_data['Married'].fillna(train_data['Married'].value_counts().idxmax(), inplace=True)
train_data['Dependents'].fillna(train_data['Dependents'].value_counts().idxmax(), inplace=True)
train_data['Self_Employed'].fillna(train_data['Self_Employed'].value_counts().idxmax(), inplace=True)
train_data['LoanAmount'].fillna(train_data['LoanAmount'].mean(skipna=True), inplace=True)
train_data['Loan_Amount_Term'].fillna(train_data['Loan_Amount_Term'].value_counts().idxmax(), inplace=True)
train_data['Credit_History'].fillna(train_data['Credit_History'].value_counts().idxmax(), inplace=True)
```

```
In [20]: #Check missing values
```

```
train_data.isnull().sum()  
train_data
```


Out[20]:

	Loan_ID	Gender	Married	Dependents	Education	Self_Employed	Applicant
0	LP001002	Male	No	0	Graduate	No	
1	LP001003	Male	Yes	1	Graduate	No	
2	LP001005	Male	Yes	0	Graduate	Yes	
3	LP001006	Male	Yes	0	Not Graduate	No	
4	LP001008	Male	No	0	Graduate	No	
5	LP001011	Male	Yes	2	Graduate	Yes	
6	LP001013	Male	Yes	0	Not Graduate	No	
7	LP001014	Male	Yes	3+	Graduate	No	
8	LP001018	Male	Yes	2	Graduate	No	
9	LP001020	Male	Yes	1	Graduate	No	
10	LP001024	Male	Yes	2	Graduate	No	
11	LP001027	Male	Yes	2	Graduate	No	
12	LP001028	Male	Yes	2	Graduate	No	
13	LP001029	Male	No	0	Graduate	No	
14	LP001030	Male	Yes	2	Graduate	No	
15	LP001032	Male	No	0	Graduate	No	
16	LP001034	Male	No	1	Not Graduate	No	
17	LP001036	Female	No	0	Graduate	No	
18	LP001038	Male	Yes	0	Not Graduate	No	
19	LP001041	Male	Yes	0	Graduate	No	
20	LP001043	Male	Yes	0	Not Graduate	No	
21	LP001046	Male	Yes	1	Graduate	No	
22	LP001047	Male	Yes	0	Not Graduate	No	
23	LP001050	Male	Yes	2	Not Graduate	No	
24	LP001052	Male	Yes	1	Graduate	No	
25	LP001066	Male	Yes	0	Graduate	Yes	
26	LP001068	Male	Yes	0	Graduate	No	

	Loan_ID	Gender	Married	Dependents	Education	Self_Employed	Applicant
27	LP001073	Male	Yes	2	Not Graduate	No	
28	LP001086	Male	No	0	Not Graduate	No	
29	LP001087	Female	No	2	Graduate	No	
...	...	...	...	...	...	...	...
584	LP002911	Male	Yes	1	Graduate	No	
585	LP002912	Male	Yes	1	Graduate	No	
586	LP002916	Male	Yes	0	Graduate	No	
587	LP002917	Female	No	0	Not Graduate	No	
588	LP002925	Male	No	0	Graduate	No	
589	LP002926	Male	Yes	2	Graduate	Yes	
590	LP002928	Male	Yes	0	Graduate	No	
591	LP002931	Male	Yes	2	Graduate	Yes	
592	LP002933	Male	No	3+	Graduate	Yes	
593	LP002936	Male	Yes	0	Graduate	No	
594	LP002938	Male	Yes	0	Graduate	Yes	
595	LP002940	Male	No	0	Not Graduate	No	
596	LP002941	Male	Yes	2	Not Graduate	Yes	
597	LP002943	Male	No	0	Graduate	No	
598	LP002945	Male	Yes	0	Graduate	Yes	
599	LP002948	Male	Yes	2	Graduate	No	
600	LP002949	Female	No	3+	Graduate	No	
601	LP002950	Male	Yes	0	Not Graduate	No	
602	LP002953	Male	Yes	3+	Graduate	No	
603	LP002958	Male	No	0	Graduate	No	
604	LP002959	Female	Yes	1	Graduate	No	
605	LP002960	Male	Yes	0	Not Graduate	No	
606	LP002961	Male	Yes	1	Graduate	No	

	Loan_ID	Gender	Married	Dependents	Education	Self_Employed	ApplicantIncome
607	LP002964	Male	Yes	2	Not Graduate	No	11368
608	LP002974	Male	Yes	0	Graduate	No	13608
609	LP002978	Female	No	0	Graduate	No	9592
610	LP002979	Male	Yes	3+	Graduate	No	13065
611	LP002983	Male	Yes	1	Graduate	No	13135
612	LP002984	Male	Yes	2	Graduate	No	13368
613	LP002990	Female	No	0	Graduate	Yes	13420

614 rows × 13 columns

```
In [21]: #Convert some object data type to int64
gender_stat = {"Female": 0, "Male": 1}
yes_no_stat = {'No' : 0, 'Yes' : 1}
dependents_stat = {'0':0, '1':1, '2':2, '3+':3}
education_stat = {'Not Graduate' : 0, 'Graduate' : 1}
property_stat = {'Semiurban' : 0, 'Urban' : 1, 'Rural' : 2}

train_data['Gender'] = train_data['Gender'].replace(gender_stat)
train_data['Married'] = train_data['Married'].replace(yes_no_stat)
train_data['Dependents'] = train_data['Dependents'].replace(dependents_stat)
train_data['Education'] = train_data['Education'].replace(education_stat)
train_data['Self_Employed'] = train_data['Self_Employed'].replace(yes_no_stat)
train_data['Property_Area'] = train_data['Property_Area'].replace(property_stat)
```

```
In [22]: #Preview data information
data.info()
data.isnull().sum()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 614 entries, 0 to 613
Data columns (total 13 columns):
Loan_ID                614 non-null object
Gender                 601 non-null object
Married                611 non-null object
Dependents              599 non-null object
Education              614 non-null object
Self_Employed          582 non-null object
ApplicantIncome         614 non-null int64
CoapplicantIncome        614 non-null float64
LoanAmount              592 non-null float64
Loan_Amount_Term        600 non-null float64
Credit_History          564 non-null float64
Property_Area           614 non-null object
Loan_Status             614 non-null object
dtypes: float64(4), int64(1), object(8)
memory usage: 62.4+ KB
```

```
Out[22]: Loan_ID          0
Gender          13
Married         3
Dependents      15
Education       0
Self_Employed  32
ApplicantIncome 0
CoapplicantIncome 0
LoanAmount      22
Loan_Amount_Term 14
Credit_History 50
Property_Area   0
Loan_Status     0
dtype: int64
```

4. Making Prediction

```
In [23]: #Separate feature and target
x = train_data.iloc[:,1:12]
y = train_data.iloc[:,12]

#make variabel for save the result and to show it
classifier = ('Gradient Boosting','Random Forest','Decision Tree','K-Nearest N
y_pos = np.arange(len(classifier))
score = []
```

```
In [24]: clf = GradientBoostingClassifier()
scores = cross_val_score(clf, x, y,cv=5)
score.append(scores.mean())
print('The accuracy of classification is %.2f%%' %(scores.mean()*100))
```

The accuracy of classification is 78.02%

```
In [25]: clf = RandomForestClassifier(n_estimators=10)
scores = cross_val_score(clf, x, y,cv=5)
score.append(scores.mean())
print('The accuracy of classification is %.2f%%' %(scores.mean()*100))
```

The accuracy of classification is 76.38%

```
In [26]: clf = DecisionTreeClassifier()
scores = cross_val_score(clf, x, y,cv=5)
score.append(scores.mean())
print('The accuracy of classification is %.2f%%' %(scores.mean()*100))
```

The accuracy of classification is 69.73%

```
In [27]: clf = KNeighborsClassifier()
scores = cross_val_score(clf, x, y,cv=5)
score.append(scores.mean())
print('The accuracy of classification is %.2f%%' %(scores.mean()*100))
```

The accuracy of classification is 61.24%

```
In [28]: clf = svm.LinearSVC(max_iter=5000)
scores = cross_val_score(clf, x, y, cv=5)
score.append(scores.mean())
print('The accuracy of classification is %.2f%%' %(scores.mean()*100))
```

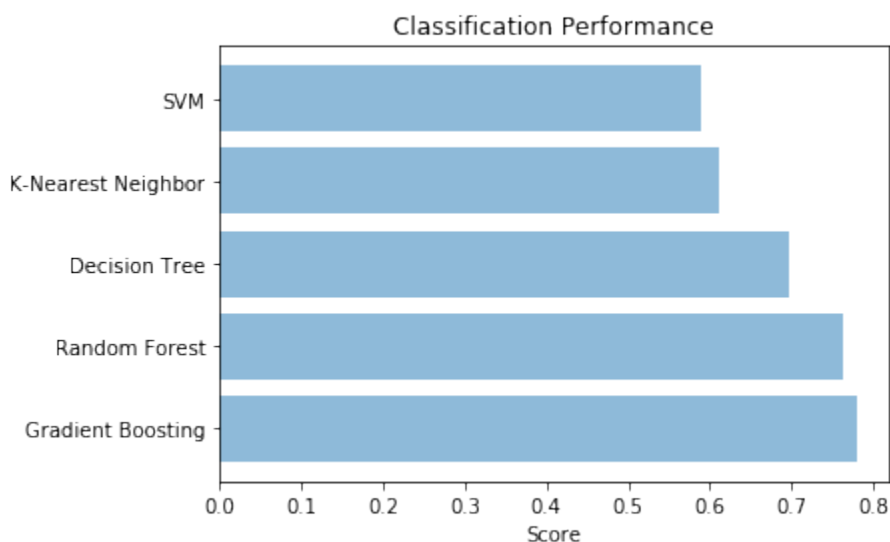
```
/opt/conda/lib/python3.6/site-packages/sklearn/svm/base.py:931: ConvergenceWarning: Liblinear failed to converge, increase the number of iterations.
    "the number of iterations.", ConvergenceWarning)
/opt/conda/lib/python3.6/site-packages/sklearn/svm/base.py:931: ConvergenceWarning: Liblinear failed to converge, increase the number of iterations.
    "the number of iterations.", ConvergenceWarning)
/opt/conda/lib/python3.6/site-packages/sklearn/svm/base.py:931: ConvergenceWarning: Liblinear failed to converge, increase the number of iterations.
    "the number of iterations.", ConvergenceWarning)
/opt/conda/lib/python3.6/site-packages/sklearn/svm/base.py:931: ConvergenceWarning: Liblinear failed to converge, increase the number of iterations.
    "the number of iterations.", ConvergenceWarning)
The accuracy of classification is 58.96%
```

```
/opt/conda/lib/python3.6/site-packages/sklearn/svm/base.py:931: ConvergenceWarning: Liblinear failed to converge, increase the number of iterations.
    "the number of iterations.", ConvergenceWarning)
```

5. Result

The result is Gradient Boosting Classifier have the highest score from other classification algorithm. These result are similar to my previous works.

```
In [29]: plt.barh(y_pos, score, align='center', alpha=0.5)
plt.yticks(y_pos, classifier)
plt.xlabel('Score')
plt.title('Classification Performance')
plt.show()
```



Reference

1. J. Heo and J. Y. Yang, "AdaBoost Based Bankruptcy Forecasting of Korean Construction Company," *Applied Soft Computing*, vol. 24, pp. 494-499, 2014.
2. C.-F. Tsai, "Feature Selection in Bankruptcy Prediction," *Knowledge Based System*, pp. 120-127, 2009.

In [30]: